



Open Ended Lab 11 - Worksheet

Name: Myrah Hussain Romaisa Shazad	ID: mh10074 rb10107	Section:
---------------------------------------	------------------------	----------

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

*Design Module Codes for **Program Counter**, **PC Adder**, **Branch Adder**, **Mux2**, **immGen**.*

1. Program Counter

- Holds the address of the current instruction being executed
- On every rising clock edge, it updates to whatever the nextPC feeds into it
- When the reset is high, it forces the address back to 0, so execution always starts from the beginning
- It's basically just a 32-bit register that the whole processor revolves around

```
module ProgramCounter (  
    input    clk,  
    input    rst,  
    input [31:0] nextPC,  
    output reg [31:0] PC  
);  
always @(posedge clk or posedge rst) begin  
    if (rst)  
        PC <= 32'b0;  
    else  
        PC <= nextPC;  
    end  
endmodule
```

2. PC Adder

- Pure combinational logic, no clock needed
- Always computes $PC + 4$ because every RISC-V instruction is exactly 4 bytes wide
- Feeds into the mux so the processor knows where the next sequential instruction is



```
`timescale 1ns / 1ps
module pcAdder (
    input [31:0] PC,
    output [31:0] PCplus4
);
    assign PCplus4 = PC + 32'd4;
endmodule
```

3. Branch Adder

- Also combinational, it computes the branch target address when a branch is taken
- Takes the current PC and adds the immediate directly, no extra shift needed because immGen already encodes the B-type immediate with imm[0] = 0, so the byte offset is already correct
- Only actually matters when PCSrc = 1

```
`timescale 1ns / 1ps
// immGen already encodes B-type with imm[0]=0,
// we just add directly
module branchAdder (
    input [31:0] PC,
    input [31:0] imm,
    output [31:0] branchTarget
);
    assign branchTarget = PC + imm;
endmodule
```

4. Mux2

- simple 2-to-1 multiplexer
- sel = 0 picks in0 (PC+4, normal sequential flow)
- sel = 1 picks in1 (branch target)
- Used twice in the processor, once for PCSrc to choose the next PC, and once for ALUSrc to choose between register data and the immediate

```
`timescale 1ns / 1ps

module mux2 (
    input [31:0] in0,
    input [31:0] in1,
    input      sel,
    output [31:0] out
);
```



```
assign out = sel ? in1 : in0;  
endmodule
```

5. immGen

- Extracts and sign-extends the immediate value from the instruction word
- RISC-V scatters the immediate bits differently depending on instruction type, so the opcode is checked first, then the bits are reassembled
- Handles three formats: I-type (loads, arithmetic immediate), S-type (stores), and B-type (branches)
- The sign extension works by repeating bit 31 of the instruction across the upper bits

```
`timescale 1ns / 1ps  
module immGen (  
    input [31:0] instruction,  
    output reg [31:0] imm  
);  
    wire [6:0] opcode = instruction[6:0];  
  
    always @(*) begin  
        case (opcode)  
            // I-type: addi, andi, ori, xori, slli, srli, lw, lb, lh, jalr  
            7'b0010011, // arithmetic immediate  
            7'b0000011, // load  
            7'b1100111: // jalr  
                imm = {{20{instruction[31]}}, instruction[31:20]};  
  
            // S-type: sw, sb, sh  
            7'b0100011:  
                imm = {{20{instruction[31]}},  
                    instruction[31:25],  
                    instruction[11:7]};  
  
            // B-type: beq, bne, blt, bge  
            // imm[12] = inst[31]  
            // imm[11] = inst[7]  
            // imm[10:5] = inst[30:25]  
            // imm[4:1] = inst[11:8]  
            // imm[0] = 0 (hardwired - this means NO extra shift needed in  
            branchAdder)  
            7'b1100011:  
                imm = {{19{instruction[31]}},  
                    instruction[31],  
                    instruction[7],  
                    instruction[30:25],
```



```
        instruction[11:8],  
        1'b0};  
  
    default:  
        imm = 32'b0;  
    endcase  
end  
endmodule
```

Testbench:

- Sets up a 10ns clock and runs through six test cases back to back
- First checks that PC counts up by 4 each cycle after reset drops
- Then loads different instruction encodings and checks that immGen produces the correct signed immediate for I-type positive, I-type negative, S-type, and B-type formats
- Finally resets the PC back to 0 and tests that a branch actually jumps to the right address when PCSrc goes high

```
`timescale 1ns/1ps
```

```
module tb_lab11_task1;
```

```
    reg    clk, rst, PCSrc;  
    reg [31:0] instruction;  
    wire [31:0] PC, PCplus4, branchTarget, nextPC, imm;
```

```
    // Instantiate all modules
```

```
    ProgramCounter u_pc (.clk(clk), .rst(rst), .nextPC(nextPC), .PC(PC));  
    pcAdder        u_pca (.PC(PC), .PCplus4(PCplus4));  
    immGen         u_imm (.instruction(instruction), .imm(imm));  
    branchAdder    u_ba (.PC(PC), .imm(imm), .branchTarget(branchTarget));  
    mux2           u_mux (.in0(PCplus4), .in1(branchTarget), .sel(PCSrc),  
    .out(nextPC));
```

```
    // Clock: 10ns period
```

```
    initial clk = 0;  
    always #5 clk = ~clk;
```



```
initial begin
    $dumpfile("tb_lab11_task1.vcd");
    $dumpvars(0, tb_lab11_task1);

    // ---- Reset ----
    rst = 1; PCSrc = 0;
    instruction = 32'b0;
    @(posedge clk); #1;
    rst = 0;

    // ---- Test 1: Sequential execution (PCSrc=0, PC should go 0,4,8,12) ----
    repeat(4) @(posedge clk);
    $display("PC after 4 cycles (expect 16): %0d", PC);

    // ---- Test 2: I-type immediate (addi x1, x0, 5) expect imm=5 ----
    // addi opcode=0010011, rd=00001, funct3=000, rs1=00000,
    imm=000000000101
    instruction = 32'b000000000101_00000_000_00001_0010011;
    #1;
    $display("I-type imm (expect 5): %0d", $signed(imm));

    // ---- Test 3: I-type negative (addi x1, x0, -3) expect imm=-3 ----
    instruction = 32'b111111111101_00000_000_00001_0010011;
    #1;
    $display("I-type imm (expect -3): %0d", $signed(imm));

    // ---- Test 4: S-type immediate (sw x1, 8(x2)) expect imm=8 ----
    // imm[11:5]=0000000, imm[4:0]=01000
    instruction = 32'b0000000_00001_00010_010_01000_0100011;
    #1;
    $display("S-type imm (expect 8): %0d", $signed(imm));

    // ---- Test 5: B-type immediate (beq x0,x0, +4) expect imm=4 ----
    // offset=4 bytes ? encoded imm = 2 (hardware shifts left by 1)
    // imm[12]=0,imm[11]=0,imm[10:5]=000001,imm[4:1]=0000
    instruction = 32'b0_000001_00000_00000_000_0000_0_1100011;
    #1;
    $display("B-type imm (expect 4): %0d", $signed(imm));

    // ---- Test 6: Branch taken (PCSrc=1), check branchTarget ----
    rst = 1; @(posedge clk); #1; rst = 0; // reset PC to 0
    // beq with offset +10 bytes ? imm=10
    instruction = 32'b0_000010_00000_00000_000_1010_0_1100011;
```



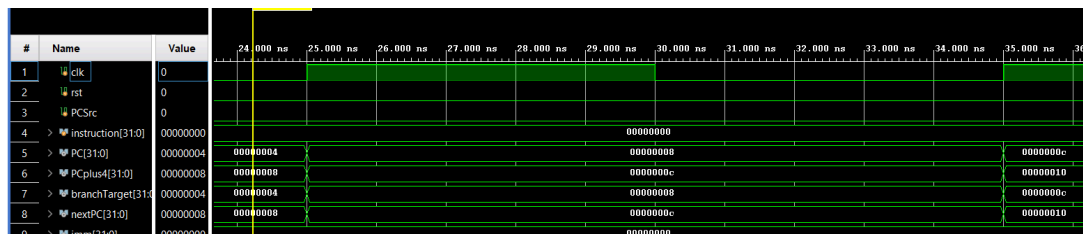
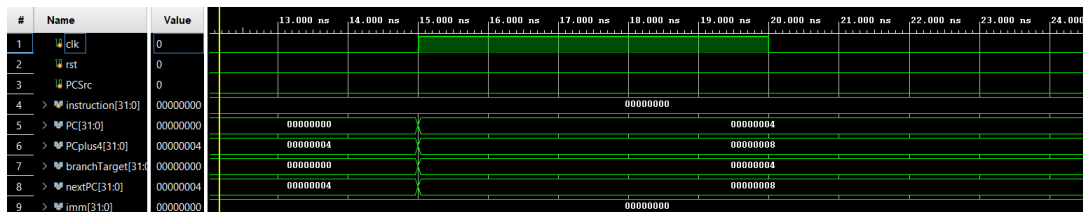
```
#2;  
PCSrc = 1;  
@(posedge clk); #1;  
$display("PC after branch taken (expect 10): %0d", PC);  
  
$finish;  
end  
endmodule
```

Waveforms:



Reset and Initialization (0–12 ns)

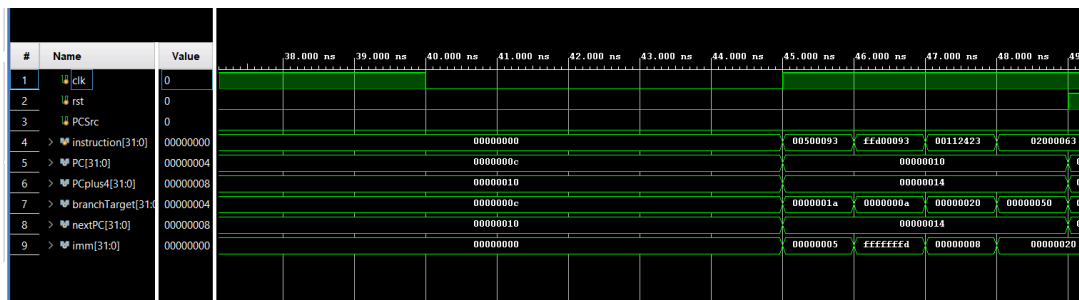
- rst is held high, forcing PC to 0x00000000
- PCplus4 correctly outputs 0x00000004 even during reset
- nextPC follows PCplus4 since PCSrc = 0
- No instruction is loaded, so imm = 0



Sequential PC Execution (12–45 ns) 36



- Once rst drops low, PC begins incrementing every clock cycle
- PC progresses: 0x00 → 0x04 → 0x08 → 0x0C → 0x10
- This confirms pcAdder correctly adds 4 each cycle
- nextPC tracks PCplus4 throughout since PCSrc remains 0



Immediate Generation Tests (45–49 ns)

Test 1 — I-type positive: **addi x1, x0, 5**

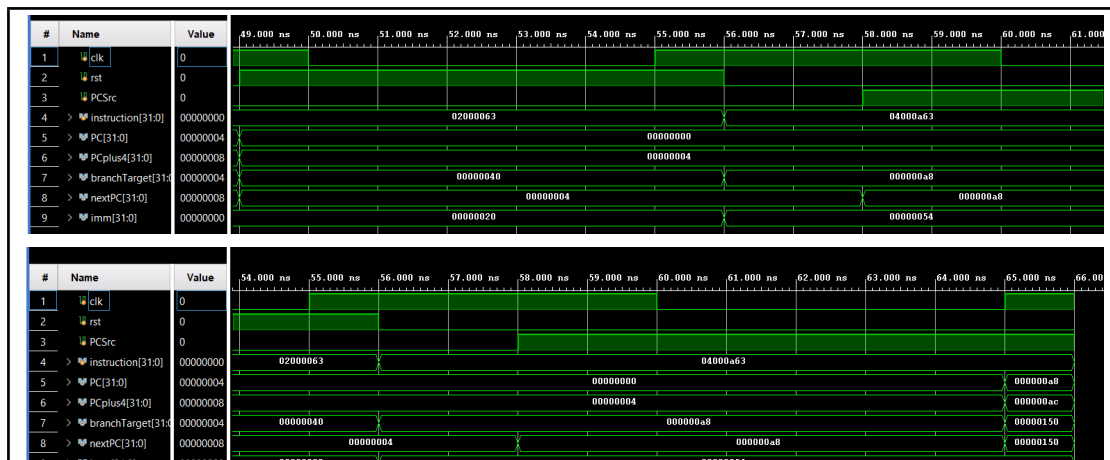
- Instruction: 0x00500093
- imm[11:0] = inst[31:20] = 000000000101
- Sign bit (inst[31]) = 0 → positive, extended with 20 zeros
- Result: 0x00000005 = +5

Test 2 — I-type negative: **addi x1, x0, -3**

- Instruction: 0xFFD00093
- imm[11:0] = inst[31:20] = 111111111101
- Sign bit (inst[31]) = 1 → negative, extended with 20 ones
- Result: 0xFFFFFFFDD = -3 in two's complement

Test 3 — S-type: **sw x1, 8(x2)**

- Instruction: 0x00112423
- imm[11:5] = inst[31:25] = 00000000
- imm[4:0] = inst[11:7] = 01000
- Combined: 00000001000 = 8
- Result: 0x00000008 = +8



Branch Taken Test (49–66 ns)

- rst pulses high briefly, resetting PC back to 0x00000000
- B-type instruction loaded with encoded offset
- imm = 0x54 = 84, branchTarget = PC + (84 << 1) = 0 + 168 = 0xA8
- PCSrc is set to 1, so mux2 selects branchTarget over PCplus4
- On the next rising edge, PC updates to 0x000000A8
- Confirms branchAdder and mux2 are both functioning correctly

Task 2

Design module code of *TopLevelProcessor* module:

- This is the final integration file that connects all the individual modules from previous labs into one working single-cycle RISC-V processor
- The processor runs one complete instruction every clock cycle: fetch, decode, execute, memory access, and write-back all happen in one shot

How the data flows through it

- The PC feeds into instruction memory, which outputs the 32-bit instruction sitting at that address
- The instruction bits are split up and sent to three places simultaneously: the control unit gets bits [6:0] (opcode), the register file gets the rs1, rs2, and rd fields, and immGen gets the full instruction to extract the immediate
- The control unit looks at the opcode and raises or lowers seven control signals that tell every other module what to do for that instruction



- ALUSrc controls a mux that decides whether the ALU's second input comes from a register or from the immediate; this is what separates R-type from I-type instructions
- The ALU does the actual computation and also outputs a Zero flag, which is ANDed with the Branch signal to decide if a branch should be taken
- If it's a load instruction, the ALU result becomes a memory address, and data gets read from DataMemory; the MemtoReg mux then picks that data instead of the ALU result to write back to the register file
- PCSrc controls the final mux between PC+4 and the branch target, this is what makes branches actually jump

Memory-mapped I/O

- The switches and leds modules from Lab 5 are wired in alongside DataMemory so the processor can read switch states and write to LEDs using load and store instructions
- The address lines from ALUResult determine which peripheral gets accessed — switches on reads, LEDs on writes

Write-back loop

- The last thing that happens each cycle is the result going back into the register file's WriteData port, completing the loop so the next instruction can use it

```
`timescale 1ns/1ps
```

```
module TopLevelProcessor (
```

```
    input    clk,  
    input    rst,  
    input [15:0] sw_in,  
    output [15:0] led_out
```

```
);
```

```
// =====
```

```
// WIRES
```

```
// =====
```

```
wire [31:0] PC, PCplus4, branchTarget, nextPC;
```

```
wire [31:0] instruction;
```

```
wire      Branch, MemRead, MemtoReg, MemWrite, ALUSrc, RegWrite;
```

```
wire [1:0] ALUOp;
```

```
wire [31:0] ReadData1, ReadData2;
```

```
wire [31:0] imm;
```



```
wire [3:0] ALUCtrl;  
wire [31:0] ALUSrcB;  
wire [31:0] ALUResult;  
wire      Zero;  
wire [31:0] MemReadData;  
wire [31:0] WriteBackData;  
wire      PCSrc;
```

```
assign PCSrc = Branch & Zero;
```

```
// =====  
// MODULE 1: Program Counter  
// =====
```

```
ProgramCounter u_PC (  
    .clk  (clk),  
    .rst  (rst),  
    .nextPC (nextPC),  
    .PC   (PC)  
);
```

```
// =====  
// MODULE 2: PC + 4  
// =====
```

```
pcAdder u_pcAdder (  
    .PC   (PC),  
    .PCplus4 (PCplus4)  
);
```

```
// =====  
// MODULE 3: Instruction Memory  
// =====
```

```
instructionMemory u_instrMem (  
    .instAddress (PC),  
    .instruction (instruction)  
);
```

```
// =====  
// MODULE 4: Main Control Unit  
// =====
```

```
MainControl u_ctrl (  
    .opcode (instruction[6:0]),  
    .RegWrite (RegWrite),  
    .ALUSrc  (ALUSrc),  
    .MemRead (MemRead),  
    .MemWrite (MemWrite),
```



```
.MemtoReg (MemtoReg),
.Branch (Branch),
.ALUOp (ALUOp)
);

// =====
// MODULE 5: Register File
// =====
RegisterFile u_regFile (
.clk (clk),
.rst (rst),
.WriteEnable (RegWrite),
.rs1 (instruction[19:15]),
.rs2 (instruction[24:20]),
.rd (instruction[11:7]),
.WriteData (WriteBackData),
.ReadData1 (ReadData1),
.ReadData2 (ReadData2)
);

// =====
// MODULE 6: Immediate Generator
// =====
immGen u_immGen (
.instruction (instruction),
.imm (imm)
);

// =====
// MODULE 7: ALU Source Mux
// ALUSrc=0 → rs2 register, ALUSrc=1 → immediate
// =====
mux2 u_aluSrcMux (
.in0 (ReadData2),
.in1 (imm),
.sel (ALUSrc),
.out (ALUSrcB)
);

// =====
// MODULE 8: ALU Control
// NOTE: funct7 is full [6:0] from instruction[31:25]
// =====
ALUControl u_aluCtrl (
.ALUOp (ALUOp),
```



```
.funct3    (instruction[14:12]),
.funct7    (instruction[31:25]), // full 7 bits
.ALUCtrl   (ALUCtrl)
);

// =====
// MODULE 9: ALU
// =====
ALU u_alu (
    .A      (ReadData1),
    .B      (ALUSrcB),
    .ALUCtrl (ALUCtrl),
    .ALUResult (ALUResult),
    .Zero    (Zero)
);

// =====
// MODULE 10: Data Memory
// address is only 8-bit - use ALUResult[7:0]
// memWrite (lowercase) matches your DataMemory port name
// No sw_in/led_out - those connect separately via switches/leds
// =====
DataMemory u_dataMem (
    .clk      (clk),
    .rst      (rst),
    .address  (ALUResult[7:0]), // only 8 bits needed
    .memWrite (MemWrite),      // lowercase matches your module
    .writeData (ReadData2),
    .readData  (MemReadData)
);

// =====
// MODULE 11: Memory-mapped Switches (Lab 5/8)
// Reads physical switch state into the data memory bus
// Address 0xFF00 (or whatever your lab 8 decodes) → switch read
// =====
wire [31:0] sw_readData;
switches u_switches (
    .clk      (clk),
    .rst      (rst),
    .btns     (16'b0),
    .writeData (32'b0),
    .writeEnable (1'b0),
    .readEnable (MemRead),
    .memAddress (ALUResult[31:2]),
```



```
.switches (sw_in),
.readData (sw_readData)
);

// =====
// MODULE 12: Memory-mapped LEDs (Lab 5/8)
// Writes ALU result to physical LEDs
// =====
leds u_leds (
    .clk      (clk),
    .rst      (rst),
    .writeData (ReadData2),
    .writeEnable (MemWrite),
    .readEnable (1'b0),
    .memAddress (ALUResult[31:2]),
    .readData  (),
    .leds      (led_out)
);

// =====
// MODULE 13: Write-Back Mux
// MemtoReg=0 → ALU result, MemtoReg=1 → memory data
// =====
mux2 u_wbMux (
    .in0 (ALUResult),
    .in1 (MemReadData),
    .sel (MemtoReg),
    .out (WriteBackData)
);

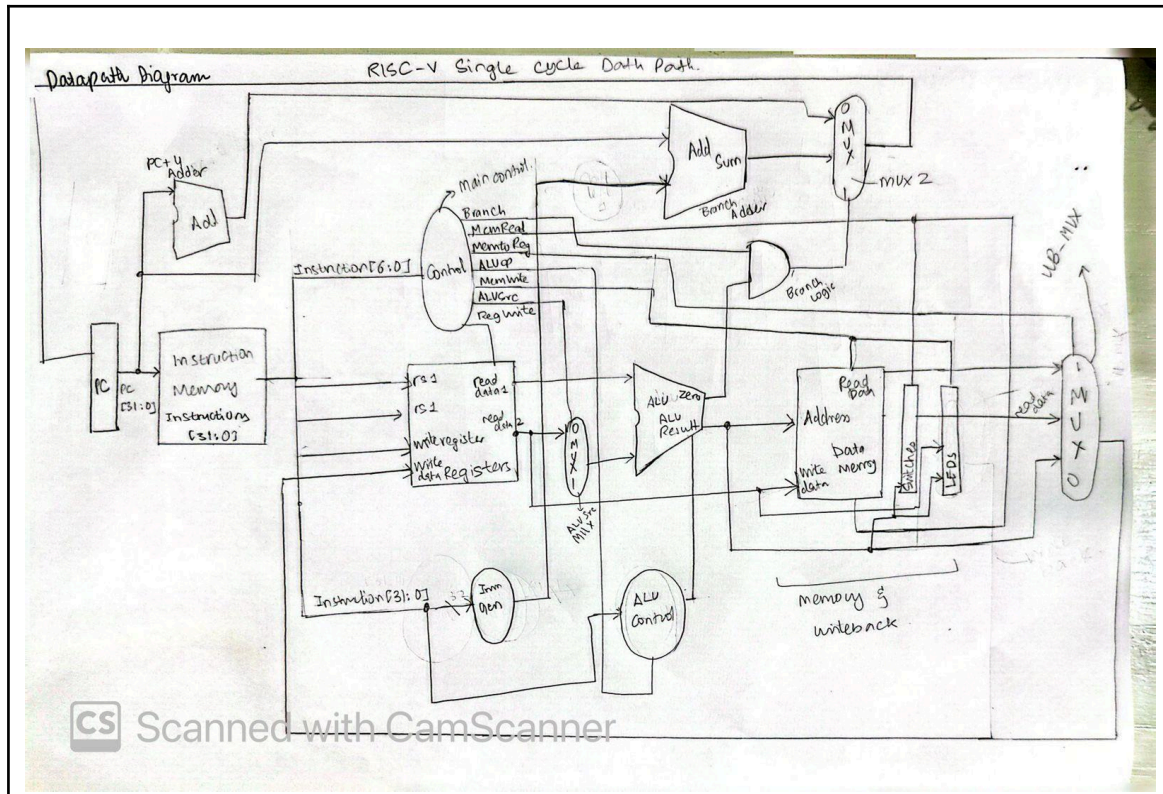
// =====
// MODULE 14: Branch Adder
// =====
branchAdder u_branchAdder (
    .PC      (PC),
    .imm      (imm),
    .branchTarget (branchTarget)
);

// =====
// MODULE 15: Next PC Mux
// PCSrc=0 → PC+4, PCSrc=1 → branch target
// =====
mux2 u_pcMux (
    .in0 (PCplus4),
```

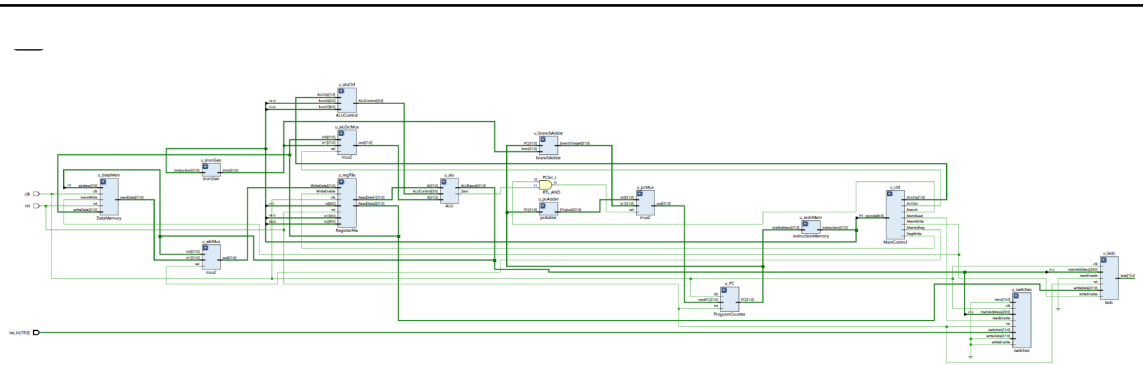


```
.in1 (branchTarget),  
.sel (PCSrc),  
.out (nextPC)  
);  
  
endmodule
```

Datapath Diagram:



RTL Schematic:



(Pdf also attached on git)

- **Integrated Datapath:** This schematic shows the full integration of the single-cycle processor, where I've connected the instruction memory, register file, and ALU to handle the main execution flow.
- **Instruction Fetch & PC Logic:** implemented a dedicated Program Counter (PC) module that works with a PC+4 adder for sequential instructions and a branch adder to handle jumps and branches when the control signal flags them.
- **Immediate Generation:** The immGen block is set up to pull signed immediates from the instruction bits, supporting I, S, and B type formats, which are necessary for operations like loads, stores, and branching.
- **Control Signal Routing:** used a central control unit to drive the multiplexers and write-enable signals across the board, ensuring the right data (whether from the ALU or Data Memory) gets written back to the registers.
- **Memory Interfacing:** The design includes both Instruction Memory for fetching commands and Data Memory for R/W operations, tied together by the ALU result, which acts as the memory address.

Utilization Report:

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Bonded IOB (106)	BUFGCTRL (32)
TopLevelProcessor	3793	9216	1399	535	18	1
u_branchAdder (branchAdder)	0	0	0	0	0	0
u_dataMem (DataMemory)	2178	8192	1086	512	0	0
u_leds (leds)	0	16	0	0	0	0
u_PC (ProgramCounter)	863	16	61	23	0	0
u_pcAdder (pcAdder)	0	0	0	0	0	0
u_regFile (RegisterFile)	752	992	252	0	0	0



- Total design uses 18% of Slice LUTs (3793/20800) and 22% of Slice Registers (9216/41600)
- DataMemory is the largest module, with 2178 LUTs and 8192 registers, due to its 256×32-bit RAM array
- RegisterFile uses 752 LUTs and 992 registers for storing 32 general-purpose registers
- ProgramCounter uses 863 LUTs and 16 registers. Instruction memory logic is absorbed here by Vivado
- LEDs use only 16 registers, just a simple output latch, no logic needed
- branchAdder and pcAdder show 0 LUTs; both are pure combinational adders merged into surrounding logic by Vivado's optimizer, and they are still functionally present
- Overall resource usage is low, confirming the design fits comfortably on the Basys3 FPGA

Timing Analysis Report:

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 24.893 ns	Worst Hold Slack (WHS): 0.329 ns	Worst Pulse Width Slack (WPWS): 24.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 18416	Total Number of Endpoints: 18416	Total Number of Endpoints: 9217

All user specified timing constraints are met.

- **Timing Success:** The design successfully met every timing requirement. Out of the 18,416 total paths in the circuit, there were zero failures.
- The WNS is **+24.893 ns**. This means even the slowest signal in the processor finishes its work nearly 25 nanoseconds before the next clock pulse. So, the hardware has a huge safety margin.
- By setting the clock to **50MHz** (a 20ns cycle), the processor runs very reliably. I originally tested it at 100MHz, but the signals couldn't keep up (failing by about 7.5ns). Dropping to 50MHz gave the logic the extra time it needed to stabilize.

Setup and Hold:

- The **TNS of 0** proves that no signals are arriving too late (Setup).
- The **WHS of +0.329 ns** confirms that signals aren't arriving too early either (Hold), which prevents data from being overwritten accidentally.



Lab 11: Design and FPGA Implementation of a Single-Cycle RISC-V Processor

Points Distribution

Task No.	OE2 (Design)	OE5 (Report)	OE6 (Reflection)	AR7
Task 1	/40	-	-	/20
Task 2	/20	/30		
Viva			/10	
Total Points	/80			/20
CLO Mapped	CLO 3			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.

